

Rapport TD5 – SR10

Qualité de code – Tests unitaires et tests de routes

Projet *Landr* – Plateforme de recrutement

Mathis Millot & Romain Pierre

Mai 2026

Table des matières

Exercice 1 – Tests unitaires du modèle utilisateur	2
Configuration et structure	2
Méthodes testées	2
Résultats de couverture	3
Exercice 2 – Tests des routes avec supertest	4
Routes publiques et API JSON	4
Routes protégées par session	4
Routes POST et flux multi-étapes	5
CRUD recruteur et candidatures	6
Rate limiting – problème et solution	7
Bilan de couverture	7
Conclusion	8

Exercice 1 – Tests unitaires du modèle utilisateur

Configuration et structure

La configuration Jest dans `package.json` active la collecte de couverture et force la fermeture du processus après les tests :

```
1 "scripts": { "test": "jest --forceExit" },
2 "jest":      { "collectCoverage": true }
```

Les tests utilisent la vraie base de données. Un utilisateur de test est créé en `beforeAll` avec un email unique (horodaté) et supprimé en `afterAll` :

```
1 const TS          = Date.now();
2 const TEST_EMAIL  = `jest_${TS}@jest.local`;
3 let   testUserId  = null;
4
5 beforeAll(async () => {
6   testUserId = await utilisateur.create(
7     'TestNom', 'TestPrenom', TEST_EMAIL, 'TestMdp123!', '0600000000'
8   );
9 });
10
11 afterAll(async () => {
12   await DB.query('DELETE FROM Candidat WHERE id_user = ?', [
13     testUserId]);
14   await DB.query('DELETE FROM Utilisateur WHERE id_user = ?', [
15     testUserId]);
16   await DB.end();
17 });
```

Méthodes testées

Pour chaque méthode, on teste le cas nominal et les cas d'erreur. Exemple avec `findAndCheckByCredentials`.

```
1 describe('findAndCheckByCredentials', () => {
2   test('retourne null si email inexistant', async () => {
3     const result = await utilisateur.findAndCheckByCredentials(
4       'inconnu@jest.local', 'mdp'
5     );
6     expect(result).toBeNull();
7   });
8
9   test('retourne null si mot de passe incorrect', async () => {
10    expect(await utilisateur.findAndCheckByCredentials(
11      TEST_EMAIL, 'mauvaismdp'
12    )).toBeNull();
13  });
14
15  test('retourne { inactive: true } si compte INACTIF', async () => {
16    await utilisateur.setStatut(testUserId, 'INACTIF');
17    expect(await utilisateur.findAndCheckByCredentials(TEST_EMAIL, '
18      TestMdp123!'))
19      .toEqual({ inactive: true });
20    await utilisateur.setStatut(testUserId, 'ACTIF');
21  });
22 });
```

Tableau récapitulatif des méthodes testées :

Méthode	Cas testés
<code>readAll()</code>	tableau non vide, utilisateur présent
<code>read(id)</code>	id valide, id inexistant
<code>findByEmail()</code>	email existant, email inconnu
<code>findAndCheckByCredentials()</code>	email inconnu, mauvais mdp, valide, inactif
<code>update()</code>	mise à jour réussie, id inexistant
<code>getDocuments()</code> / <code>addDocument()</code>	liste vide, ajout et récupération
<code>setPhoto()</code>	mise à jour de la photo
<code>setStatut()</code>	passage INACTIF puis ACTIF
<code>findOrCreateByGoogle()</code>	création, récupération existant, inactif
<code>createRecruteur()</code>	sans organisation, avec organisation
<code>changeRole()</code>	vers Admin, vers Recruteur, réaffectation FK

Résultats de couverture

Fichier	Stmts	Branches	Fonctions	Lignes
<code>utilisateur.js</code>	95%	85%	100%	96%

Exercice 2 – Tests des routes avec supertest

Les tests de routes utilisent la vraie base de données. Après installation de **supertest**, les tests sont placés dans `test/route.test.js`. Trois utilisateurs de test (candidat, admin, recruteur) sont créés dans le `beforeAll` global et supprimés dans `afterAll`.

Routes publiques et API JSON

Les routes sans authentification sont testées directement avec `request(app)` :

```
1 test('GET / repond 200 avec du HTML', async () => {
2   const res = await request(app).get('/');
3   expect(res.statusCode).toBe(200);
4   expect(res.headers['content-type']).toMatch(/html/);
5 });
6
7 describe('GET /offres', () => {
8   test('repond 200 sans filtre', async () => {
9     const res = await request(app).get('/offres');
10    expect(res.statusCode).toBe(200);
11  });
12   test('repond 200 avec filtre q', async () => {
13     const res = await request(app).get('/offres?q=developpeur');
14     expect(res.statusCode).toBe(200);
15   });
16 });
```

Pour la route JSON `/api/organisation/check`, on vérifie le contenu du corps :

```
1 test('retourne { exists: false } si aucun siren', async () => {
2   const res = await request(app).get('/api/organisation/check');
3   expect(res.headers['content-type']).toMatch(/json/);
4   expect(res.body).toEqual({ exists: false });
5 });
6
7 test('retourne { exists: true, nom } si organisation trouvee', async ()
8   => {
9   const res = await request(app)
10     .get('/api/organisation/check?siren=${TEST_SIREN}');
11   expect(res.body.exists).toBe(true);
12   expect(res.body.nom).toBe('OrgRouteTest');
13 });
```

Routes protégées par session

Sans session, les routes protégées renvoient une redirection :

```
1 test('GET /admin redirige vers /connection', async () => {
2   const res = await request(app).get('/admin');
3   expect(res.statusCode).toBe(302);
4   expect(res.headers.location).toBe('/connection');
5 });
```

Avec un mauvais rôle, `isAdmin` renvoie un 403 :

```
1 test('GET /admin repond 403 pour un candidat connecte', async () => {
2   const res = await candidatAgent.get('/admin');
```

```

3   expect(res.statusCode).toBe(403);
4 });

```

Pour tester les routes avec session, on utilise `request.agent(app)` qui maintient les cookies entre les requêtes :

```

1 describe('Routes candidat avec session', () => {
2   test('GET /profil_candidat repond 200', async () => {
3     const res = await candidatAgent.get('/profil_candidat');
4     expect(res.statusCode).toBe(200);
5   });
6
7   test('POST /modifier_profil redirige vers /profil_candidat', async ()
8     => {
9     const res = await candidatAgent.post('/modifier_profil')
10      .type('form')
11      .send({ nom: 'Nouveau', prenom: 'Prenom',
12            email: CANDIDAT_EMAIL, num_tel: '0700000000' });
13     expect(res.statusCode).toBe(302);
14     expect(res.headers.location).toBe('/profil_candidat');
15   });
16 });

```

Routes POST et flux multi-étapes

Les routes POST d'inscription testent la validation des données et les appels aux méthodes du modèle :

```

1 describe('POST /inscription/etape1', () => {
2   test('redirige si mot de passe trop court', async () => {
3     const res = await request(app).post('/inscription/etape1')
4       .type('form').send({ email: 'a@a.fr', mdp: 'court', confirm: '
5       court' });
6     expect(res.headers.location).toContain('mdp-court');
7   });
8
9   test('redirige si email deja utilise (appelle findByEmail)', async ()
10     => {
11     const res = await request(app).post('/inscription/etape1')
12      .type('form')
13      .send({ email: CANDIDAT_EMAIL, mdp: 'mdp12345', confirm: 'mdp12345
14      ' });
15     expect(res.headers.location).toContain('error=email');
16   });
17
18   test('redirige vers /informations_personnelles si donnees valides',
19     async () => {
20     const res = await request(app).post('/inscription/etape1')
21      .type('form')
22      .send({ email: 'route_${TS}@jest.local', mdp: 'mdp12345', confirm:
23      'mdp12345' });
24     expect(res.headers.location).toBe('/informations_personnelles');
25   });
26 });

```

Le flux multi-étapes est testé avec un agent qui conserve la session entre les étapes :

```

1 describe('Flux inscription candidat (etapes 2 et 3)', () => {
2   const agent = request.agent(app);
3   let createdUserId = null;
4
5   afterAll(async () => {
6     if (createdUserId) {
7       await DB.query('DELETE FROM Candidat WHERE id_user = ?', [
8         createdUserId]);
9       await DB.query('DELETE FROM Utilisateur WHERE id_user = ?', [
10        createdUserId]);
11     }
12   });
13
14   test('etape2 stocke en session et redirige', async () => {
15     await agent.post('/inscription/etape1').type('form')
16       .send({ email: INS_EMAIL, mdp: 'mdp12345', confirm: 'mdp12345' });
17     const res = await agent.post('/inscription/etape2').type('form')
18       .send({ nom: 'Nom', prenom: 'Prenom', num_tel: '0600000000' });
19     expect(res.headers.location).toBe('/profil_professionnel');
20   });
21
22   test('etape3 cree l\'utilisateur et redirige', async () => {
23     const res = await agent.post('/inscription/etape3').type('form').
24       send({});
25     expect(res.headers.location).toBe('/profil_candidat');
26     const user = await utilisateur.findByEmail(INS_EMAIL);
27     if (user) createdUserId = user.id_user;
28   });
29 });

```

CRUD recruteur et candidatures

Pour tester les routes recruteur de gestion de fiches et d'offres, une organisation de test est insérée directement en base dans le `beforeAll` :

```

1 beforeAll(async () => {
2   await DB.query(
3     'INSERT INTO Organisation (siren, nom, type, siege_social,
4       validation,
5       id_admin_createur) VALUES (?, ?, ?, ?, ?, ?)',
6     [SIREN_REC_CRUD, 'OrgRecCRUD', 'Entreprise', '', 'OUI', adminId]
7   );
8   await DB.query(
9     "INSERT INTO Appartient (siren_organisation, id_recruteur, statut)
10     VALUES (?, ?, 'ACCEPTEE')", [SIREN_REC_CRUD, recruteurId]
11   );
12 });

```

Les tests CRUD s'enchaînent en réutilisant les IDs créés par les tests précédents :

```

1 let ficheId = null;
2 let offreId = null;
3
4 test('POST /recruteur/fiches/nouvelle cree une fiche', async () => {
5   const res = await recruteurAgent.post('/recruteur/fiches/nouvelle')
6     .type('form').send({ intitule: 'Dev', ..., siren_organisation:
7     SIREN_REC_CRUD });

```

```

7   expect(res.statusCode).toBe(302);
8   const [[fiche]] = await DB.query(
9     'SELECT id_fiche FROM FicheDePoste WHERE siren_organisation = ?
10    ORDER BY id_fiche DESC LIMIT 1', [SIREN_REC_CRUD]
11  );
12  ficheId = fiche.id_fiche;
13 });
14
15 test('GET /recruteur/fiches/:id/modifier repond 200', async () => {
16   expect(ficheId).not.toBeNull();
17   const res = await recruteurAgent.get(`/recruteur/fiches/${ficheId}/
18     modifier`);
19   expect(res.statusCode).toBe(200);
20 });

```

Pour la route POST /candidature qui utilise Multer, les données sont envoyées en multipart avec .field() :

```

1 test('POST /candidature cree et affiche la confirmation', async () => {
2   const res = await candidatAgent.post('/candidature')
3     .field('id_offre', String(offreIdCand));
4   expect(res.statusCode).toBe(200);
5 });
6
7 test('POST /candidature redirige si deja candidate', async () => {
8   const res = await candidatAgent.post('/candidature')
9     .field('id_offre', String(offreIdCand));
10  expect(res.statusCode).toBe(302);
11  expect(res.headers.location).toBe('/profil_candidat');
12 });

```

Rate limiting – problème et solution

La route POST /login bloque les connexions après 10 tentatives par IP sur 15 minutes. Avec plusieurs groupes de tests créant chacun leur propre agent et leur propre connexion, ce seuil était dépassé, causant des échecs en cascade (réponse /connection?error=ratelimit).

La solution : trois agents partagés connectés une seule fois dans le beforeAll global, réutilisés par tous les groupes :

```

1 const candidatAgent = request.agent(app);
2 const adminAgent = request.agent(app);
3 const recruteurAgent = request.agent(app);
4
5 beforeAll(async () => {
6   // creation des utilisateurs...
7   await candidatAgent.post('/login').type('form')
8     .send({ email: CANDIDAT_EMAIL, mdp: TEST_MDP });
9   await adminAgent.post('/login').type('form')
10    .send({ email: ADMIN_EMAIL, mdp: TEST_MDP });
11   await recruteurAgent.post('/login').type('form')
12    .send({ email: REC_EMAIL, mdp: TEST_MDP });
13 });

```

Bilan de couverture

Résultats après exécution de la suite complète (116 tests, tous passants) :

Fichier	Stmts	Branches	Fonctions	Lignes
utilisateur.js	95%	85%	100%	96%
candidature.js	100%	100%	100%	100%
demande_recuteur.js	100%	100%	85%	100%
organisation.js	83%	100%	64%	84%
fiche_de_poste.js	80%	54%	80%	83%
offre.js	89%	69%	100%	89%
routes/index.js	73%	71%	81%	79.9%
upload.js	46%	10%	0%	47%
app.js	61%	0%	0%	63%
Total	76%	67%	74%	80.5%

upload.js et app.js sont en dessous de 80% : leurs lignes non couvertes sont des handlers de filtres Multer et des callbacks OAuth/Passport qui nécessiteraient un vrai flux d'upload ou d'authentification OAuth pour être déclenchés.

Exercice	Fichier cible	Lignes	Objectif
Ex. 1 – Modèle user	model/utilisateur.js	96%	≥ 80%
Ex. 2 – Routes	routes/index.js	79.9%	≥ 80%

Conclusion

La suite de tests mise en place couvre les deux niveaux demandés : 26 tests unitaires sur le modèle utilisateur (`model.test.js`) et 87 tests d'intégration sur les routes (`route.test.js`). Les points notables sont la gestion du rate limiting par partage d'agents, l'usage de `.field()` pour les routes Multer, et le nettoyage rigoureux des données de test par `afterAll` dans le bon ordre selon les contraintes de clé étrangère.